

RemSolution — Addendum : améliorations non listées

Complément au plan de développement. Ces points ne figurent pas dans votre liste et sont ressortis de la lecture du code. Classés par gravité réelle, pas par taille.

Avertissement de lecture : je n'ai listé que ce que j'ai vérifié dans le code. Plusieurs choses que j'aurais soupçonnées sont en réalité bien faites (arabe + RTL complets, numérotation légale des factures, cache des réglages d'agence, remboursements) et n'apparaissent donc pas ici.

Bloquants métier ou légaux

A.1 ● Aucune gestion de la TVA sur les factures — 4 j

Il n'y a **aucune notion de taxe dans tout le code** : ni taux, ni montant HT/TTC, ni mention légale. La facture additionne le prix de location et les services additionnels, et c'est tout.

Or une facture tunisienne doit porter le montant hors taxe, le taux de TVA, le montant de TVA, le total TTC, le timbre fiscal, et le matricule fiscal de l'agence. **Une facture sans ces éléments n'est pas une facture** au sens comptable : elle n'est pas déductible pour un client professionnel, et l'agence ne peut pas la remettre à son comptable.

C'est, à mon avis, le trou fonctionnel le plus sérieux du produit — plus grave que tout ce qui figure dans votre liste, parce qu'il touche le seul document que le client emporte et que le fisc regarde.

À développer : matricule fiscal et taux de TVA par défaut sur l'agence (dans `AgencySettings`), décomposition HT / TVA / TTC calculée et **figée sur la facture** (jamais recalculée après émission — le taux peut changer par la loi), timbre fiscal en montant fixe paramétrable, et les variables correspondantes dans les modèles de documents (`facture.montantHT`, `facture.tauxTVA`, `facture.montantTVA`, `facture.timbre`, `agence.matriculeFiscal`).

Point d'attention : décider dès maintenant si le tarif journalier saisi par l'agence est HT ou TTC. C'est une décision irréversible en pratique — la changer plus tard fausse tout l'historique.

A.2 ● Aucune date d'expiration sur les documents clients — 1,5 j

`Client` stocke pour la CIN, le passeport et le permis : le numéro, la **date de délivrance**, le lieu et le pays. Il n'y a **aucune date d'expiration**.

Conséquence directe : rien n'empêche de louer un véhicule à un client dont le permis est périmé. En cas d'accident, l'assurance peut refuser sa garantie, et la responsabilité remonte vers l'agence qui a remis les clés. Le risque n'est pas informatique, il est financier et juridique.

À développer : `CINExpiryDate`, `PasseportExpiryDate`, `DrivingLicenceExpiryDate`, un avertissement visible dans la fiche client et **au moment de créer une location** (avertissement bloquant ou confirmation explicite, à arbitrer), et un type de notification pour les permis qui expirent bientôt — l'infrastructure de notification existe déjà, c'est un cas de plus.

Performance — le vrai risque à l'échelle

A.3 ● Aucun index sur `Renting` ni `Reservation` pour la disponibilité — 1 j

C'est la trouvaille la plus concrète de cette relecture.

Le contrôle de disponibilité (`AvailabilityChecker`) exécute, pour chaque vérification :

```
WHERE CarId = @carId AND StartDate < @end AND EndDate > @start
```

Or les seuls index existants sur ces tables sont ceux générés par `HasAgencyTenant` :

- `Renting` : `IX(AgencyId, RentingState)`
- `Reservation` : `IX(AgencyId, Status, ExpiresAt)`

Aucun ne commence par `CarId` et aucun ne porte les dates. SQL Server n'a donc que deux choix : un seek sur `(AgencyId, State)` suivi d'un balayage de toutes les locations de cette agence dans cet état, ou un balayage complet. Le coût croît linéairement avec l'historique de l'agence — pas avec le nombre de voitures.

Invisible aujourd'hui (bases de test), douloureux à 20 000 locations, inacceptable sur le marketplace où la recherche publique fait ces anti-jointures pour **chaque voiture candidate de chaque agence proche**. C'est exactement l'index prévu par la tâche 2.5 de votre feuille

de route, jamais posé.

Script idempotent :

```
IF NOT EXISTS (SELECT 1 FROM sys.indexes
               WHERE name = 'IX_Rentings_CarId_Dates' AND object_id = OBJECT_ID('dbo.Rentings'))
BEGIN
    CREATE INDEX IX_Rentings_CarId_Dates
    ON dbo.Rentings (CarId, StartDate, EndDate)
    INCLUDE (RentingState, AgencyId);
END;

IF NOT EXISTS (SELECT 1 FROM sys.indexes
               WHERE name = 'IX_Reservations_CarId_Dates' AND object_id = OBJECT_ID('dbo.Reservations'))
BEGIN
    CREATE INDEX IX_Reservations_CarId_Dates
    ON dbo.Reservations (CarId, StartDate, EndDate)
    INCLUDE (Status, AgencyId);
END;
```

À passer par une migration EF plutôt qu'à la main, pour que le *snapshot* reste cohérent. Vérifier ensuite le plan d'exécution : on attend un *index seek* sur les deux, plus un *lookup* nul grâce aux colonnes incluses.

Remarque : `CarId` étant nullable, l'index reste utile mais un filtre `WHERE CarId IS NOT NULL` réduirait sa taille. À mesurer, pas à décider d'avance.

A.4 🎯 Le contrôle de disponibilité fait deux allers-retours par vérification — 0,5 j

`AvailabilityChecker` exécute deux `AnyAsync` successifs (locations, puis réservations). Dans une vérification unitaire c'est négligeable ; dans la recherche marketplace parcourant N voitures, cela double le nombre d'allers-retours. Fusionnable en une seule requête par `UNION ALL / EXISTS` combiné. À faire **après** A.3 : les index changent complètement l'arbitrage, et optimiser avant de les poser reviendrait à optimiser le mauvais problème.

Exploitation

A.5 🎯 Aucune limitation de débit sur les points d'entrée publics — 1 j

Ni `AddRateLimiter` ni `UseRateLimiter` dans tout le projet Web. La recherche marketplace et l'inscription sont **anonymes**, donc :

- la recherche est une requête spatiale avec anti-jointures, ouverte au monde et gratuite à appeler en boucle : un seul script suffit à saturer la base pour toutes les agences à la fois ;
- l'inscription permet la création de comptes en masse et l'envoi de courriels de vérification depuis votre domaine — ce qui abîme votre réputation d'expéditeur et fait tomber en spam les courriels légitimes de toutes les agences.

Ce point figurait dans la feuille de route (6.5) et n'a pas été fait. Il est peu coûteux et devrait accompagner la mise en ligne du marketplace, pas la suivre.

A.6 🎯 Supervision : les briques sont là, rien ne surveille — 1,5 j

`Serilog.Sinks.ApplicationInsights` et `Microsoft.ApplicationInsights` sont référencés, les journaux sont structurés et enrichis (agence, utilisateur, corrélation), et un point de santé existe. Ce qui manque : les **alertes**. Personne n'est prévenu si les jobs Hangfire cessent de tourner, si le taux d'erreur monte, si le disque de la base se remplit, ou si les courriels ne partent plus.

En multi-locataire, l'absence d'alerte signifie que c'est un client qui vous apprend la panne — souvent des heures après son début. À faire : alertes sur taux d'erreur, échec de job récurrent, échec du point de santé, et latence des requêtes.

A.7 🎯 Envoi de courriels : SMTP direct, sans suivi de délivrabilité — 1 j

`SmtpEmailSender` envoie directement en SMTP. Cela fonctionne, mais il n'y a ni file de reprise, ni gestion des rejets (*bounces*), ni trace de délivrabilité. Un rappel de réservation qui n'arrive jamais est indistinguable d'un rappel arrivé.

Comme le produit s'appuie sur le courriel pour les rappels clients, les invitations à l'espace client et les mots de passe temporaires, cela mérite un prestataire (Brevo, Postmark, SES) plutôt qu'un serveur SMTP nu — surtout combiné à A.5, puisque l'inscription ouverte peut faire blacklister votre domaine. À traiter en même temps que l'Outbox (§4.6 du plan) : c'est le même chemin technique.

Métier — trous fonctionnels que votre liste ne couvre pas

A.8 Impossible de planifier une immobilisation à l’avance — 2 j

Le statut d’une voiture est un état **instantané** : **Actif** / **En entretien** / **Inactif** . Il n’existe aucune notion de **période** d’indisponibilité.

Conséquences concrètes : on ne peut pas déclarer « cette voiture part au garage du 20 au 24 » ; il faudrait s’en souvenir le 20 et basculer le statut à la main. Rien n’empêche donc de réserver aujourd’hui une voiture pour une période où elle sera au garage. Et une fois le statut basculé sur **En entretien** , les réservations déjà prises sur ces dates existent toujours — le statut ne les annule pas.

À développer : une table de périodes d’indisponibilité (**CarUnavailability(CarId, From, To, Reason)**) intégrée au prédicat de disponibilité au même titre que les locations et les réservations. Le statut instantané reste utile pour « en ce moment » ; la période règle « à cette date ».

Le même index que A.3 sert cette table — à concevoir ensemble.

A.9 La caution n’a pas de sort explicite au retour — 1 j

DepositAmount est perçue puis reportée à la conversion, et **Payment.IsRefund** permet d’enregistrer un remboursement. Mais **aucune étape ne demande, au retour, ce que devient la caution** : rendue en entier, partiellement retenue pour dommage, ou entièrement retenue.

En pratique un collaborateur doit s’en souvenir et saisir un remboursement à la main. C’est exactement le genre d’oubli qui produit un litige client trois semaines plus tard, sans trace pour l’arbitrer.

À traiter en même temps que les frais supplémentaires au retour (\$3.2 du plan) : c’est le même écran, le même moment, et la retenue de caution est en pratique le mode de paiement de ces frais.

A.10 Rien ne relie une location au conducteur autorisé, côté assurance — 0,5 j (décision) / 2 j (mise en œuvre)

Renting a un **Client** et un **SecondClient** , ce qui couvre le conducteur additionnel. Mais rien ne vérifie que le second conducteur possède un permis enregistré et valide, alors que c’est précisément lui l’angle mort d’un dossier d’assurance.

Point à arbitrer plus qu’à développer : soit on accepte que le second conducteur soit une simple mention, soit on exige un dossier client complet pour lui — et dans ce cas la validation de A.2 doit s’appliquer aux deux.

Récapitulatif et intégration au plan

	Point	Gravité	Effort
A.1	TVA / mentions légales sur factures	⦿ Légal	4 j
A.2	Dates d’expiration des documents clients	⦿ Juridique	1,5 j
A.3	Index de disponibilité manquants	⦿ Performance	1 j
A.5	Limitation de débit points publics	⦿ Sécurité	1 j
A.8	Périodes d’immobilisation planifiées	⦿ Métier	2 j
A.6	Alertes de supervision	⦿ Exploitation	1,5 j
A.9	Sort de la caution au retour	⦿ Métier	1 j
A.7	Prestataire d’envoi de courriels	⦿ Exploitation	1 j
A.4	Fusion des requêtes de disponibilité	⦿ Performance	0,5 j
A.10	Second conducteur / assurance	⦿ Décision	0,5 à 2 j
	Total		13,5 à 15,5 j ≈ 3 semaines

Où les insérer dans le plan

- **A.3 rejoint le Lot 1** (débloquer la production). Un index d’une journée qui évite un balayage complet sur la table la plus lue n’attend pas — et il est bien plus facile à poser maintenant, sur des tables presque vides, que sur des tables chargées où sa création prend un verrou.
- **A.1 et A.2 constituent un nouveau lot « Conformité », à placer juste après le Lot 1.** Ce sont les deux seuls points de tout le projet qui exposent l’agence — et donc vous — à autre chose qu’un client mécontent.
- **A.5, A.6, A.7 rejoignent le Lot 4** (exploitation et mise en ligne).

- **A.8, A.9, A.4 rejoignent le Lot 3** (manques fonctionnels), A.9 étant à fusionner avec la tâche 3.2.
- **A.10 est une décision à ajouter à la §6** du plan, pas une tâche.

Effet sur le total

Périmètre	Durée
Plan initial, lots 1 à 6	38,5 j
Addendum (hors A.10 en option haute)	+13,5 j
Total révisé, hors mobile et GPS	52 j ≈ 10,5 semaines

Ce que je retiens

Le produit est bien construit — l’isolation multi-locataire, la machine à états des réservations, la gestion du temps UTC, la suppression logique sélective et la collation insensible aux accents sont d’un niveau qu’on ne voit pas souvent dans un projet de cette taille.

Les trois choses qui l’empêchent d’être vendable ne sont pas des fonctionnalités manquantes, ce sont trois oublis structurels : **on ne peut pas se connecter à une agence neuve** (§4.1 du plan), **la facture n’est pas une facture légale** (A.1), et **une base perdue est perdue** (§4.3). Aucun des trois n’est difficile. Tous les trois sont éliminatoires.